计算机组成原理实验报告

计算机学院 吴文晖 25325278

实验主题：寄存器堆与 ALU

1.实验原理

本实验由 32 位 ALU、32×32 位寄存器堆以及连接两者的数据通路组成。ALU 对两个 32 位操作数执行加法、减法、按位与、按位或、对 A 按位取反和有符号小于比较；寄存器堆具有两个组合读口和一个同步写口；联合模块从寄存器堆读出两个操作数，将结果送入 ALU，并把外部装载数据或 ALU 计算结果写回指定寄存器。

ALU 的 3 位操作码 ALUctr 依次选择上述六项运算：000 至 101 分别对应加法、减法、按位与、按位或、对 A 取反和有符号比较。加法和减法保留低 32 位，有符号比较使用$signed 将两个操作数按补码解释。测试台设置 A=3、B=5，每 20 ns 切换一次操作码，因此可以在同一段 0 至 120 ns 波形中核对六项结果。

表 2-1　alu32 六项运算预期结果

时间/ns	ALUctr	运算	Result（十六进制）
0-20	000	A+B	00000008
20-40	001	A-B	FFFFFFFE
40-60	010	A&B	00000001
60-80	011	A|B	00000007
80-100	100	~A	FFFFFFFC
100-120	101	有符号 A<B	00000001

寄存器堆内部用 reg [31:0] registers [0:31]保存 32 个 32 位寄存器。写操作在 Clk 上升沿执行，只有 rst=0、RegWr=1 且写地址 Rw 不为 0 时才写入 busW；复位时全部寄存器清零。读操作不等待时钟沿，busA 和 busB 直接由 Ra、Rb 选中的寄存器内容决定，其中 0 号寄存器的读出值恒为 0。

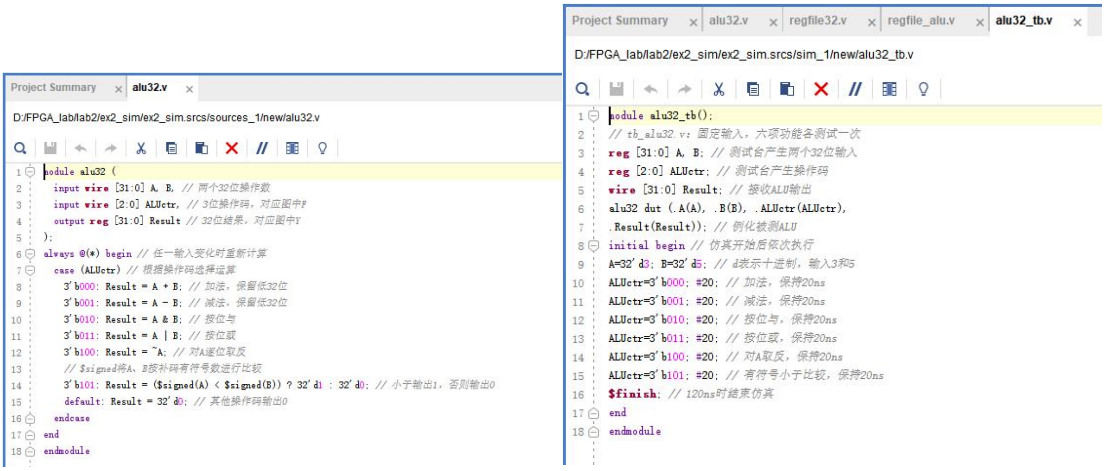
联合模块以 LoadData 控制写入来源：busW = LoadData ? DataIn : Result。LoadData=1 时，可以把外部数据分别写入 R9 和 R10；LoadData=0 时，R9、R10 作为 ALU 的两个输入，计算结果经 busW 写回 R8。读回 R8 的值即可确认写入

和保存均已完成。

2.仿真结果与分析

2.1 alu32 仿真

将 tb_alu32 设为仿真顶层，在波形窗口中加入 A、B、ALUctr 和 Result。A、B 与 Result 的显示进制设为十六进制，ALUctr 设为二进制，重启仿真后运行至 120 ns，波形如图所示。



alu32 代码

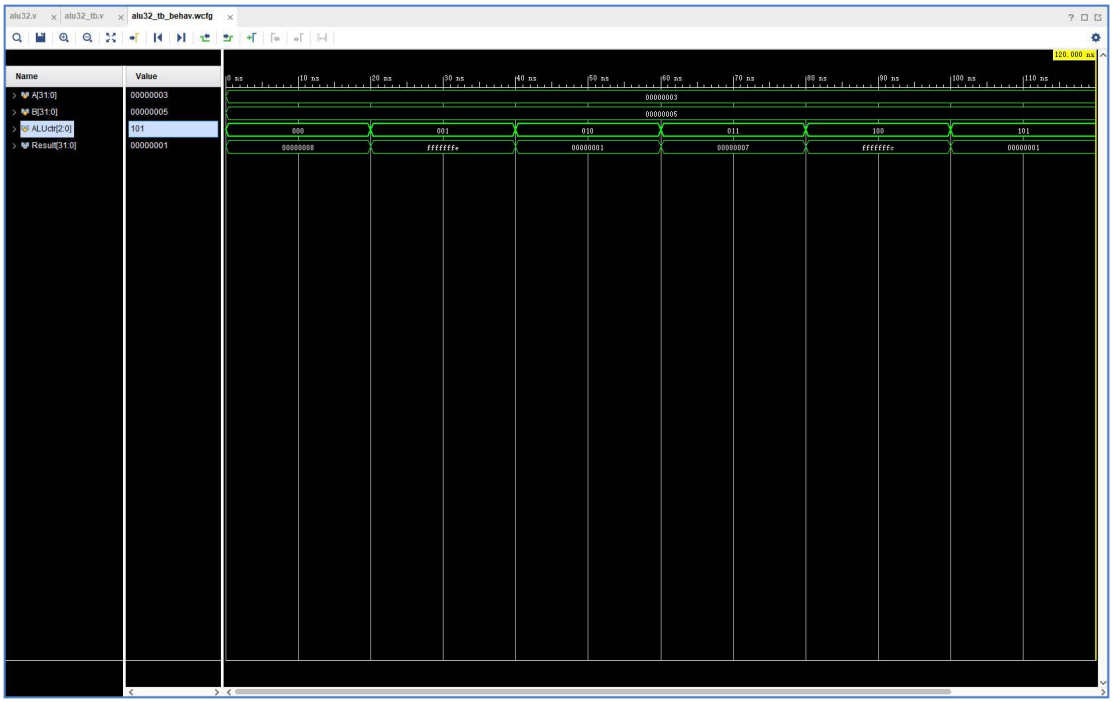


图 2-1　alu32 六项运算仿真波形

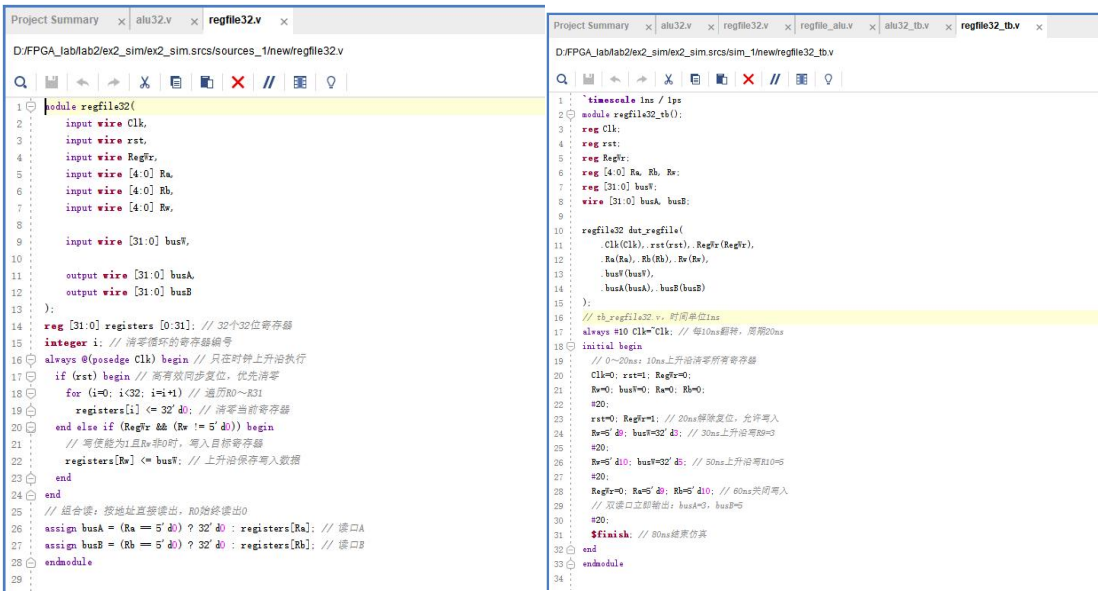
波形在 0～20 ns 显示 00000008，对应 3+5；20 至 40 ns 显示 FFFFFFFE，对

应 3-5，按 32 位补码解释为-2；40 至 60 ns 显示 00000001，对应 3&5；60 至 80 ns 显示 00000007，对应 3|5；80 至 100 ns 显示 FFFFFFFC，对应按位取反后的-4；100 至 120 ns 显示 00000001，说明有符号比较得到 3<5。

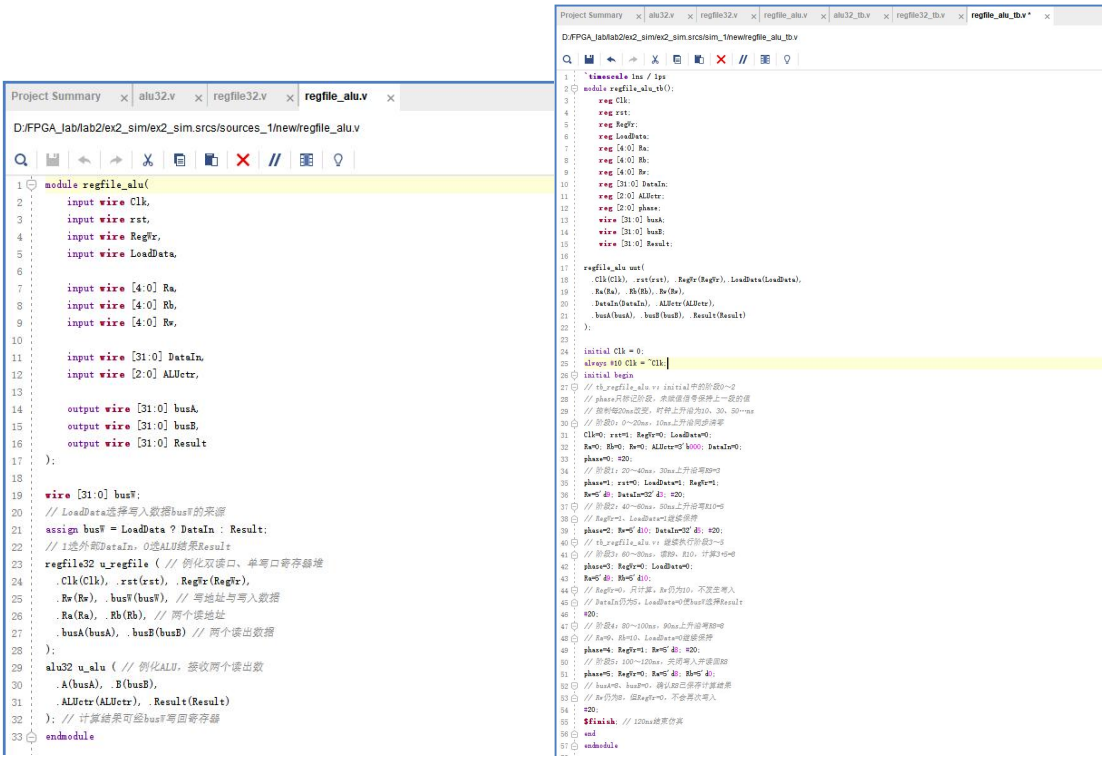
六段结果与表 2-1 逐一相同，说明操作码译码、32 位组合运算和默认分支均按设计工作。减法和取反后的高位 F 是 32 位补码的符号扩展结果，并不表示运算错误；比较结果为 1 位真值，因此高 31 位补 0。

2.2 寄存器堆与联合模块仿真

regfile_alu 波形同时覆盖装载、读取、计算、写回和读回过程。时间轴展开到 0 至 120 ns，重点保留 rst、RegWr、LoadData、Rw、Ra、Rb、busA、busB、Result 和 busW，以便逐段说明操作数来自哪个寄存器、计算结果写入哪个寄存器。



regfile 代码



```
module regfile_alu(
    input wire Clk,
    input wire rst,
    input wire RegWr,
    input wire LoadData,

    input wire [4:0] Ra,
    input wire [4:0] Rb,
    input wire [4:0] Rw,

    input wire [31:0] DataIn,
    input wire [2:0] ALUctr,

    output wire [31:0] busA,
    output wire [31:0] busB,
    output wire [31:0] Result
);

wire [31:0] busW;
// LoadData选择写入数据busW的来源
assign busW = LoadData ? DataIn : Result;
// 1选外部DataIn，0选ALU结果Result
regfile32 u_regfile ( // 例化双读口、单写口寄存器堆
  .Clk(Clk), .rst(rst), .RegWr(RegWr),
  .Rw(Rw), .busW(busW), // 写地址与写入数据
  .Ra(Ra), .Rb(Rb), // 两个读地址
  .busA(busA), .busB(busB) // 两个读出数据
);
alu32 u_alu ( // 例化ALU，接收两个读出数
  .A(busA), .B(busB),
  .ALUctr(ALUctr), .Result(Result)
); // 计算结果可经busW写回寄存器
endmodule
```

```
`timescale 1ns / 1ps
module regfile_alu_tb();
    reg Clk;
    reg rst;
    reg RegWr;
    reg LoadData;
    reg [4:0] Ra;
    reg [4:0] Rb;
    reg [4:0] Rw;
    reg [31:0] DataIn;
    reg [2:0] ALUctr;
    reg [2:0] phase;
    wire [31:0] busA;
    wire [31:0] busB;
    wire [31:0] Result;

regfile_alu uut(
    .Clk(Clk), .rst(rst), .RegWr(RegWr), .LoadData(LoadData),
    .Ra(Ra), .Rb(Rb), .Rw(Rw),
    .DataIn(DataIn), .ALUctr(ALUctr),
    .busA(busA), .busB(busB), .Result(Result)
);

initial Clk = 0;
always #10 Clk = ~Clk;
initial begin
// tb_regfile_alu.v: initial中的阶段0~2
// phase只标记阶段，求稳值信号保持上一段的值
// 时钟每20ns改变，时钟上升沿为10、30、50…ns
// 阶段0：0~20ns，10ns上升沿同步清零
Clk=0; rst=1; RegWr=0; LoadData=0;
Ra=0; Rb=0; Rw=0; ALUctr=3'b000; DataIn=0;
phase=0; #20;
// 阶段1：20~40ns，30ns上升沿写R9=3
phase=1; rst=0; LoadData=1; RegWr=1;
Rw=5'd9; DataIn=32'd3; #20;
// 阶段2：40~60ns，50ns上升沿写R10=5
// RegWr=1、LoadData=1继续保持
phase=2; Rw=5'd10; DataIn=32'd5; #20;
// tb_regfile_alu.v: 继续执行阶段3~5
// 阶段3：60~80ns，读数，R10、计算3+5=8
phase=3; RegWr=0; LoadData=0;
Ra=5'd9; Rb=5'd10;
// RegWr=0，只计算，Rw仍为10，不发生写入
// DataIn仍为5，LoadData=0使busW选择Result
#20;
// 阶段4：80~100ns，90ns上升沿写R9=8
// Rw=9、Rb=10、LoadData=0继续保持
phase=4; RegWr=1; Rw=5'd9; #20;
// 阶段5：100~120ns，无对寄存器堆写
phase=5; RegWr=0; Ra=5'd9; Rb=5'd9;
// busA=8、busB=8，我认知已保存计算结果
// Rw仍为9，但RegWr=0，不会再次写入
#20;
$finish; // 120ns结束仿真
end
endmodule
```

regfile_alu 代码

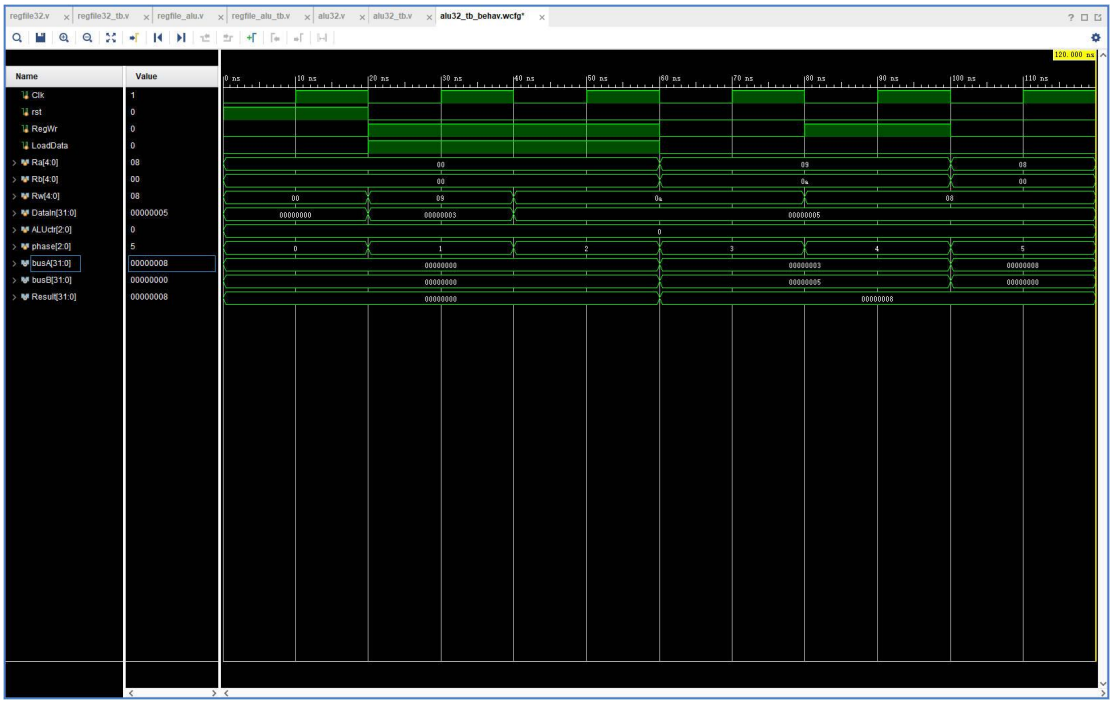


图 2-2　寄存器读取、ALU 计算与结果写回波形

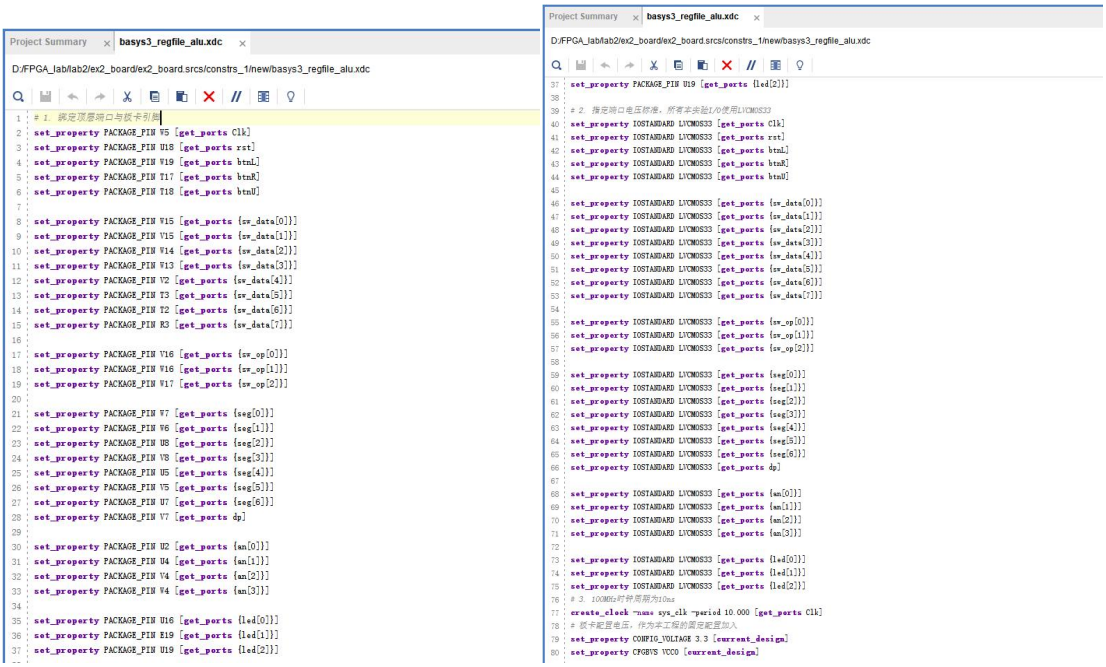
0～20 ns 的 rst=1 在 10 ns 上升沿完成寄存器清零。30 ns 时，RegWr=1、LoadData=1、Rw=9、DataIn=3，R9 写入 00000003；50 ns 时，Rw=10、DataIn=5，R10 写入 00000005。到 60 ns，Ra=9、Rb=10 且 RegWr=0，组合读口立即给出 busA=00000003、busB=00000005，ALU 输出 Result=00000008，此时只计算而不

写回。

90 ns 时，LoadData=0、RegWr=1、Rw=8，busW 选择 Result，R8 在上升沿写入 00000008。100 至 120 ns 关闭写使能并把 Ra 改为 8，busA 读回 00000008、busB 读回 00000000。读回值与 ALU 结果一致，说明双读口、单写口、写入来源选择和写回路径能够协同工作。

3.上板验证

上板工程以 basys3_regfile_alu_top 为顶层。SW11 至 SW4 提供 8 位补码数据，高 24 位复制符号位后形成 32 位 input_value；SW3 至 SW1 选择操作码。BTNL、BTNR 和 BTNU 经消抖后分别触发“写 R9”“写 R10”“计算并写回 R8”，BTNC 用于复位。LED0 至 LED2 分别表示 A 已装载、B 已装载和结果已写回，四位数码管在计算前预览当前输入，计算后锁定显示 R8 低 16 位。



xdc 文件

https://github.com/shimmer256/computer-organization-lab/tree/main/lab2

视频 2-1　寄存器完成整数运算的上板演示（链接内含 4 个视频）

演示以 A=3、B=5 为例：按 BTNC 复位后输入 00000011 并按 BTNL，LED0 点亮，说明 R9 已保存 00000003；输入 00000101 并按 BTNR，LED1 点亮，说明 R10 已保存 00000005；设置操作码 000 并按 BTNU，R8 写入并读回 00000008，

数码管显示 0008，LED2 点亮。修改拨码后显示仍保持 0008，说明 Result 已保存在 R8 并由结果显示路径锁定。

继续切换 SW3 至 SW1，可以逐项检查减法显示 FFFE、按位与显示 0001、按位或显示 0007、对 A 取反显示 FFFC、有符号比较显示 0001。32 位运算结果完整保存在 R8 中，但板上的四位数码管只显示低 16 位，因此高位全 F 的负数会以末四位的形式出现，例如-2 保存为 FFFFFFFE，屏幕显示 FFFE。

A=3、B=-5，A=-3、B=5 以及 A=-3、B=-5 的情况均在链接视频内有所体现，不再用文字一一赘述。

4.实现中的检查与处理方法

有符号比较先确认 A、B 两侧都使用$signed，避免一侧按有符号数、另一侧按无符号数解释。对于 3 和 5 这类正数，两种解释结果相同，但换成负数后会出现相反结论，因此波形中应同时检查操作数和比较结果。

寄存器写回检查始终围绕 rst=0、RegWr=1、Rw≠0 和时钟上升沿四个条件。0 号寄存器不写入；若写使能没有在正确阶段置 1，R8 会保持旧值，读回的 busA 便不能等于 Result。

写回阶段需要确认 busW 由 LoadData 正确选择。装载 R9、R10 时应选择 DataIn，计算阶段应选择 Result；同时 RegWr 必须在计算阶段先关闭，待 Rw 切换为 8 后再打开，避免把旧数据重复写入错误地址。

显示检查分两步进行：先用 busA 从 R8 完整读回 32 位结果，再截取 busA[15:0] 送数码管。这样既能确认 R8 确实保存了计算结果，也能解释数码管为什么只显示低 16 位。

5.实验结论

本次实验完成了 32 位 ALU、32×32 位寄存器堆以及寄存器堆与 ALU 联合模块的设计和仿真。alu32 波形覆盖六项运算，输出依次符合 00000008、FFFFFFFE、00000001、00000007、FFFFFFFC 和 00000001；联合波形在 60 ns 读出 R9=3 与 R10=5 并得到 8，在 90 ns 写 R8，在 100 至 120 ns 成功读回 8。

上板验证进一步检查了补码输入扩展、按键单次触发、寄存器装载、ALU 运算、R8 写回和结果锁定显示。仿真结果与板上操作流程一致，说明数据能够从 R9、R10 正确读出，ALU 结果能够写回 R8 并读回，数字显示、LED 状态与复位流程满足实验要求。